RESUME · BACKEND ENGINEER

이승하 · Seungha Lee

분산 시스템의 데이터 일관성과 운영 가능한 구조를 설계·구축하는 **7년차 백엔드 엔지니어**.

shalwxy@gmail.com · GitHub · LinkedIn · Blog

이 문서는 요약본입니다. 프로젝트별 상세·설계 결정·도식은 경력기술서 (career)에 있습니다.

MAPLESTORY UNIVERSE · 최근 3년

3년	6	4	5
재직	주요 Go 서비스	신규 서비스·도메인·봇	배포 대역 dev·test·qa·stage·live

INTRODUCE

서비스가 어떻게 요청을 받고, 데이터를 어떻게 보관하며, 어떤 경로로 실패와 복구를 만들어 내는지에 집중해 왔습니다. Java 백엔드(2년) → AI/GPU 인프라 R&D(2년) → Go 백엔드(3년)의 **7년**을 거치며, 분산 시스템의 데이터 일관성과 운영 가능한 구조를 설계·구축했습니다.

최근 3년은 Maplestory Universe에서 두 축에 집중했습니다. 체인 간 자산 이동을 다루는 **크로스체인 전송 도메인**을 처음부터 설계했고, 서비스마다 흩어져 있던 제어 로직을 **단일 정책 서비스**로 모으는 일을 주도했습니다. 둘 다 "여러 소스에서 오는 상태를 어떻게 한 곳에서 일관되게 다루는가"라는 같은 질문 위에 있고, 그 위에서 운영을 자동화하는 **CDC 분석 파이프라인**과 **LLM 기반 운영 봇**을 만들었습니다.

최종 업데이트 · 2026. 10. 30.

SKILL

3 Production 설계·운영 가능 · 2 운영 경험 있음 · 1 기초 수준

Languages	3 Go 3 Java (Spring)	2 Python 3 SQL	2 Shell
Communication	3 gRPC 3 HTTP / REST 3 SSE	2 Envoy 3 Kafka 2 Debezium (CDC)	2 Watermill
Data	3 PostgreSQL 3 Redis	2 MongoDB 2 DocumentDB	2 Snowflake
Infrastructure	3 Docker 3 Kubernetes (EKS) 3 Helm	3 GitLab CI 2 ArgoCD 2 Rundeck	2 Ansible
Observability	3 Datadog 2 Jaeger	2 OpenSearch 2 Prometheus·Grafana	

EXPERIENCE

NEXON UNIVERSE · Backend Developer · Platform Development Team

2023.11 ~ 현재

NEXON 자회사로, Maplestory IP를 기반으로 한 Web3 게임 플랫폼 **Maplestory Universe**를 만들고 있습니다. 크로스체인 전송 도메인(Swap & Warp)과 정책 서비스(api-sentinel)를 신설·주도했고, 거래 이력·결제·알림·데이터 파이프라인·CI/CD에 함께 참여했습니다.

UCLICK · Backend Developer → AI Engineering R&D

2019.11 ~ 2023.10

2019.11–2021.12 자사 B2B 백엔드(UNO-GATEWAY · UNO-MSP · MDB 등)를 Java/Spring으로 개발했습니다.

2022.01–2023.10 NVIDIA Omniverse R&D(IsaacSim/ROS)와 GPU 솔루션 운영을 담당했습니다.

PROJECT

아래 프로젝트는 모두 **NEXON UNIVERSE · Maplestory Universe**에서의 작업입니다.

정책 서비스 신설 · api-sentinel

2026.01 ~ 현재

4개 서비스에 흩어져 있던 접근 제어·속도 제한·잠금·기능 토글을 단일 정책 서비스로 수렴. 설계부터 마이그레이션, 배포 자동화까지 전 과정 주도.

문제

ACL, Rate Limit, Lockout, Feature Flag가 서비스마다 다른 코드와 다른 저장소에 구현되어 있었습니다. 같은 정책을 바꾸려면 서비스 4곳을 각각 고쳐 배포해야 했고, 동작이 서로 어긋나는 일이 잦았습니다.

설계

- 정책 서비스를 신설해 4개 서비스가 **단일 진입점**으로 정책을 조회하도록 구조 변경. 기존 제어 로직을 서비스별로 걷어내며 마이그레이션
- **4개 서비스 담당자와 리뷰·합의로 범위를 맞춘 뒤 단계적 전환** — 각 서비스가 기존 코드를 걷어내고 sentinel 호출로 바꾸는 PR을 단일 서비스 단위로 함께 머지
- 코드 분기 대신 **런타임 설정 기반**으로 정책을 바꾸는 체계 도입. 설정은 부팅 시 주입하는 원칙을 세워 요청 경로에서의 설정 조회·패닉 가능성을 차단
- ACL: 지갑/IP 화이트·블랙리스트 · Rate Limit: Sliding/Fixed Window + jitter · Lockout: OTP 반복 실패 차단, 실패 횟수 유지/초기화를 정책으로 설정화 · Feature Flag: 런타임 변경

결과

정책의 소스 오브 트루스가 한 곳으로 모이면서 서비스 간 동작 불일치가 구조적으로 사라졌고, 신규 서비스는 제어 로직을 다시 만들 필요 없이 sentinel에 붙기만 하면 되는 상태가 됐습니다.

`Go` `gRPC` `PostgreSQL` `Redis`

PostgreSQL CDC → Snowflake 파이프라인

2026.01 ~ 2026.11

배치 동기화와 애플리케이션 Kafka 발행에 기대던 분석 데이터 적재를 Debezium 기반 CDC로 교체. 발행 실패로 유실되던 경로를 없애고 실시간성을 확보. 설계·구현 담당.

문제 PostgreSQL → Snowflake 적재는 주기 배치였고, 거래 확정 이벤트는 애플리케이션이 Kafka에 직접 발행했습니다. 발행이 실패하면 DB에는 남아 있지만 하류에는 영원히 도착하지 않는 데이터가 생겼고, 배치 주기만큼의 지연은 분석과 운영 조회를 느리게 만들었습니다.

설계
- Debezium 커넥터로 PostgreSQL 변경을 Kafka로 스트리밍하고, Snowflake Stream/Task로 적재. **DB 커밋 자체가 발행**이 되어 애플리케이션 발행 실패라는 유실 경로가 사라짐
- replication slot이 방치되면 WAL이 쌓여 운영 DB를 위협하므로, slot **keepalive 배치**를 두어 유휴 구간에서도 slot이 전진하도록 관리
- Snowflake Task를 조건부로 실행해 변경이 없는 구간에 웨어하우스 비용이 발생하지 않도록 최적화

결과 적재 지연이 **일 단위 배치 주기에서 커밋 직후**로 줄었고, Snowflake Task 조건부 실행으로 유휴 구간 비용도 함께 축소.

`Debezium` `PostgreSQL replication slot` `Kafka` `Snowflake Stream/Task`

에러시로봇 · LLM 기반 운영 봇

2026.01 ~ 2026.11

Slack에 쏟아지는 에러 알림을 LLM이 먼저 읽고 스레드에 원인·관련 변경·대응 포인터를 답하는 봇. 운영 에러의 1차 분류를 사람에서 봇으로 넘김.

문제 운영 에러 알림은 사람이 하나씩 열어 로그를 찾고 최근 배포와 대조해야 했습니다. 알림 수가 늘수록 1차 분류에 드는 시간이 대응 속도를 결정했고, 야간·주말에는 그마저 지연됐습니다.

설계
- 에러 알림 수신 → LLM 1차 분석 → Slack 스레드 자동 응답 흐름. 분석 시 **GitLab 최근 커밋 대조**와 **OpenSearch 히스토리**를 함께 넣어 "무엇이 바뀌어서 터졌는지"까지 답하게 구성
- 봇이 판단하기 어려운 케이스는 담당자에게 **escalation**하도록 해, 자동 응답이 오답을 확신하는 경로를 차단

결과 **1차 분류가 사람 → 봇**으로 넘어가 야간·주말 대응 지연이 해소됐고, 반복되는 에러는 과거 분석을 재사용해 즉시 포인터를 제공.

`Vertex AI (LLM)` `Slack API` `GitLab` `OpenSearch`

대용량 트랜잭션 이력 재정비 · 크로스체인 전송 도메인 신설

2024.06 ~ 2025.09

거래·민팅·전송 이력을 쌓는 대용량 서비스(api-activity)의 레이어·컨슈머·메시지 포맷을 재정비하고, 그 위에 크로스체인 전송 도메인(Swap & Warp)과 BFF를 처음부터 신설. 서로 다른 두 이벤트 소스가 만드는 순서 불일치와 중복을 일관성 있게 흡수하는 적재 경로를 만듦.

문제 Maplestory Universe의 거래·민팅·전송 이력은 하나의 서비스에 쌓이지만, 요청 측(tracker)과 체인 측(watcher) 두 소스에서 별도 이벤트가 순서 보장 없이 도착하고, 같은 블록 시각의 이벤트가 중복으로 올 수 있었습니다. 그 위에 추가되는 **크로스체인 자산 전송(Swap & Warp) 이력을 추적·조회하는 도메인은 아예 없었습니다.**

설계
- **api-activity 레이어 재정비 · 컨슈머 일반화 · 메시지 포맷 v2**— 이후 거래/민팅/전송/Swap&Warp/USDT가 같은 적재 경로 위에 차례로 올라감
- Swap & Warp를 거래 이력과 분리된 **별도 도메인**으로 정의하고, 게이트웨이 역할의 신규 BFF(bff-swapnwarp) 구성
- 두 소스 이벤트를 묶는 **상관관계 키**를 명시적으로 정하고, 조회 키와 잠금 시드를 동일하게 맞춰 "임의 row 매칭"이 구조적으로 불가능하도록 설계
- 요청 정보(Info)와 결과(Result)의 동시 적재를 **Redis NX + PostgreSQL advisory lock**으로 직렬화, **DB 제약 전파로 롤백 → 재시도 → 멱등 Modify 수렴 경로** 확보, 비정상 이벤트는 DLQ로 분리

결과 크로스체인 전송 이력이 사용자 관점에서 한 줄로 보이는 조회 기능을 라이브까지 제공했고, 이후 **USDT·워프 지연 같은 확장이 모두 같은 적재 경로 위에 얹혔습니다.**

`Go` `gRPC` `Kafka` `PostgreSQL` `Redis` `MongoDB` `DocumentDB`

그 외 참여

- **결제·인증** · Worldpay(FraudSight 3DS) 신용카드 인증·재인증 · **Redis Lua**로 OTP 일일 3회 카운트 증가+만료 원자 처리 · 스테이블 코인(USDT) 선차감·롤백을 토큰별로 일반화
- **통합 알림·동기화** · MSU 통합 알림(SSE)에서 JWT 검증 자체 흡수, 대량 수신자를 **gRPC streaming 500건 청크·5만 건 상한**으로 분리 · 캐릭터 민트 닉네임 중복 Redis 분산 락 선점
- **운영 자동화·마이그레이션** · KYC 업체 변경에 따른 **전 사용자 데이터 이관 배치**(Argos → Sumsub, 구간 분할 병렬·재진입 가능한 커서·호출 한도 관리, live 완료) · 디스코드 뱃지·역할 동기화 배치
- **인프라·CI/CD·게이트웨이** · 담당 서비스의 GitLab CI · Dockerfile · Helm/K8s 매니페스트 · DDL 운영 · Envoy gRPC → REST route · 인증 설정 · **전사 proto 저장소 도메인별 모듈화 주도**

HOW I WORK

- **실패 경로부터 정합니다.** 선차감·롤백, DLQ·재시도, CDC처럼 "잘못됐을 때 어떻게 복구할지"를 먼저 정하고 구현합니다. 결함 수정은 피처와 분리해 먼저 머지합니다 — 피처가 롤백돼도 수정은 살아남아야 하니까요.
- **정책은 코드 분기가 아니라 설정으로 받습니다.** 정책이 바뀔 때 커밋이 아니라 설정이 바뀌어야 하고, 설정은 요청 경로가 아니라 부팅 시 주입합니다.
- **담당 서비스의 배포·매니페스트·DDL은 제 일입니다.** GitLab CI → Helm/Kubernetes → Rundeck 배포, 대역마다 다른 배포 방식, DBA 검토를 거친 DDL까지 한 사람이 끝까지 봅니다.
- **요청 하나를 끝까지 따라갈 수 있어야 합니다.** 상관관계 키를 먼저 설계하고, Datadog·Jaeger·OpenSearch에서 실제로 추적되는지 배포 후 확인합니다.
- **리뷰에서 나온 지적은 팀 원칙으로 승격합니다.** 결제·보안 기능은 리뷰 2회차까지 돌리고, "외부 호출은 log-and-swallow 금지", "설정은 부팅 시 주입" 같은 원칙을 문서화해 다음 작업에 반영합니다.

© 2026 Seungha Lee · shalwxy@gmail.com